

Modül 15: Optional

`Optional` tek bir iş için var: **meşru olarak hiçbir şey bulamayabilecek bir metodun dönüş tipi olmak**. Bunun için kullanıldığında mükemmeldir. Başka bir yerde kullanıldığında genellikle yerine geçtiği null değerinden daha kötüdür, çünkü güvenlik eklemekten tören ekler.

Bu modül çoğunlukla bu farkla ilgili.

Neler Öğreneceksiniz

- `Optional` size gerçekte ne kazandırıyor ve bunu kimin için kazandırıyor
- Asla "orada mı" diye sormayan işlemler
- `map` ile `flatMap`, stream'lerdekiyle aynı kural
- `orElse` ile `orElseGet` neden birbirinin yerine geçmez
- Çökmeyi geri getiren dahil beş yanlış kullanım

Başka Bir Dilden Geliyorsanız

	Başka yerlerde	Java
Yok olan değer	<code>None</code> , <code>null</code> , <code>undefined</code>	<code>Optional.empty()</code>
Güvenli erişim	<code>?.</code>	<code>.map(...)</code>
Varsayılan	<code>x ?? y</code> , <code>or</code>	<code>.orElse(y)</code>
Derleyici zorlaması	Rust <code>Option</code> , Kotlin <code>T?</code>	yok

Son satır önemli. Rust ya da Kotlin'de derleyici yokluğu göz ardı etmenizi engeller. Java'nın `Optional` sınıfı sıradan bir sınıftır ve hiçbir şey `.get()` çağırmanızı engellemez. Bu, **bir tipe desteklenen bir gelenektir**, garanti değil.

Ders

Ne işe yarar

İki imzayı karşılaştırın:

```
User findUser(int id) // might return null. Read the source to find out.
Optional<User> findUser(int id) // says so, in the type.
```

Tüm değer önerisi budur. **Çağırının yükümlülüğünün görünür olmasıyla** ilgilidir, içeride null'dan kaçınmakla değil.

`Optional.of` bilerek null'ı reddeder. Onu yazmak değerin orada olduğunu iddia etmek demektir, dolayısıyla bir NPE iddianızın yanlış olduğunu söyler. Gerçekten bilmiyorsanız `ofNullable` kullanın.

Kutudan çıkarmadan kullanmak

Deyimsel biçim asla bir değerin var olup olmadığını sormaz. Onunla ne yapılacağını ve neye geri düşüleceğini tarif eder. `OptionalBasics.java` dosyasından:

```
findUser(1).map(User::email).orElse("no email")
findUser(1).filter(u -> u.name().startsWith("a")).map(User::name).orElse("no match")
findUser(9).or(() -> findUser(1))           // an alternative Optional, so it chains
```

`ifPresentOrElse` iki dallı durumu ele alır. `Optional.stream()` sıfır ya da bir eleman verir, dolayısıyla `flatMap(Optional::stream)` bir boru hattı içinde ayrı bir filtre olmadan ıskalamaları düşürür.

map ile flatMap

Stream'lerdekiyle aynı kural. Fonksiyonunuz düz bir değer döndürüyorsa `map` kullanın. Zaten bir `Optional` döndürüyorsa `map` size iç içe bir tane verir:

```
map with an Optional-returning fn: Optional[Optional[ada@example.com]]
flatMap:                           Optional[ada@example.com]
```

Bir zincirde aniden iç içe `Optional` tipleri belirdiğinde `flatMap` istiyordunuz.

orElse ile orElseGet

En yaygın `Optional` hatası. `OrElseVsOrElseGet.java` dosyasından, iki çağrı da **dolu** bir `Optional` üzerinde yapılmış:

```
calling orElse:
>>> expensiveDefault ran, called from orElse
result: the real value
calling orElseGet:
result: the real value
```

İkisi de gerçek değeri döndürdü. Yalnızca biri, sonra attığı varsayılanı hesapladı.

	Aldığı	Değerlendirilme
<code>orElse(T other)</code>	bir değer	her zaman, çünkü Java argümanları çağrıdan önce değerlendirir
<code>orElseGet(Supplier<T>)</code>	bir lambda	yalnızca boşken

Varsayılanın yan etkisi olduğu anda performans notu olmaktan çıkar:

```
after orElse on a PRESENT optional, counter = 1
after orElseGet on the same,             counter = 0
```

Artık bir doğruluk hatası. Satır ekleyen, istek gönderen ya da loglayan bir varsayılan, yapmaması gerektiğinde yapacaktır.

Kural: `orElse("")` ve `orElse(0)` sorun değil. `orElse(buildDefault())` ve `orElse(repo.findDefault())` hatadır.

`orElseThrow(supplier)`, "burada yokluk bir hatadır" demenin dürüst yoludur, ve neyin beklendiğini açıklayan bir mesaj yazmanızı sağlar.

Beş yanlış kullanım

`AntiPatterns.java` dosyasından:

1. `isPresent()` sonra `get()`. Bu, daha fazla yazımla yapılmış bir null kontrolüdür. Böyle çiftlerin neredeyse hepsi `map`, `filter`, `ifPresent`, `ifPresentOrElse`, `orElse` ya da `orElseGet` olur.
2. Alan olarak `Optional`. `Serializable` değildir, örnek başına bir nesneye mal olur ve bir alan zaten null olabilir. Alanı düz tutun ve erişimciden bir `Optional` döndürün.
3. Parametre olarak `Optional`. Artık her çağırın `Optional.of(x)` ya da `Optional.empty()` yazmak zorunda, ve yine de null geçirebilirler, dolayısıyla hiçbir şey kazanmadınız. Bir aşırı yükleme kullanın.
4. `Optional<List<T>>`. Çağırının artık üç durumu var: yok, var-ve-boş, var-ve-dolu. Boş bir liste zaten burada hiçbir şey yok demektir.
5. Sıcak döngüde `Optional`. Her biri bir tahsistir. Bu, o argümanın gerçekten tuttuğu ender yerlerden biri.

Önlemesi gereken çökme

`UnguardedGet.java`:

```
Exception in thread "main" java.util.NoSuchElementException: No value present
```

Bu bir `NullPointerException`'dan daha kötüdür, daha iyi değil. Java 14'ten beri bir NPE null olan tam ifadeyi isimlendirir. "No value present" hiçbir şey isimlendirmez. Ve kod *güvenli görünür*, çünkü tipte `Optional` geçer, dolayısıyla inceleyen kişi üstünden atlar.

`get()` metodunun `orElseThrow()` metodunun daha iyi karşılamadığı hiçbir kullanımı yoktur. JDK 10 argümansız `orElseThrow()` metodunu tam olarak bunun için ekledi: aynı davranış, ama bir gözden kaçma değil bilinçli bir seçim olarak okunuyor.

Çalıştırın

```
java modules/15-optional/examples/OptionalBasics.java
java modules/15-optional/examples/OrElseVsOrElseGet.java
java modules/15-optional/examples/AntiPatterns.java

# Bilerek başarısız olur.
java modules/15-optional/examples/UnguardedGet.java

./scripts/verify-examples.sh modules/15-optional
```

Sık Yapılan Hatalar

Hesaplanan varsayımlarla `orElse`. Dolu ya da boş, her seferinde çalışır.

`isPresent()` ardından `get()`. Kostüm giymiş bir null kontrolü yazdınız.

`Optional` alanlar ve parametreler. Hiçbiri amaç değildi, ikisi de gürültü ekler.

`Optional<List<T>>` döndürmek. Boş liste döndürün.

`get()` çağırmak. Mesajlı `orElseThrow` kullanın.

`Optional` döndüren bir metottan `null` döndürmek. Oluyor, ve her iki dünyanın en kötüsü: çağıran tipe güveniyor ve yine de bir NPE alıyor.

Özet Çıkarımlar

- `Optional` aramalar için bir dönüş tipidir, genel amaçlı bir null sarmalayıcı değil.
- Bir gelenektir, garanti değil. Java'nın derleyicisi onu zorlamaz.
- Ne yapılacağını tarif edin, asla orada olup olmadığını sormayın.
- Fonksiyon zaten bir `Optional` döndürüyorsa `flatMap`.
- `orElse` argümanını her zaman değerlendirir; `orElseGet` değerlendirmez. Yan etkiyle bu bir doğruluk hatasıdır.
- Asla `get()`. Mesajlı `orElseThrow` ne beklediğinizi söyler.

Ödev

[homework/README.md](#)

Bir arama zinciri kurun, `orElse` tuzağını bir sayaçla kanıtlayın ve dört yanlış kullanımı düzeltin. Referans çözüm [solutions/15-optional/](#) altında.